

Programação de Sistemas

Programação com ponteiros em C

Marcelo de Gomensoro Malheiros



Recapitulando...

» Passagem de parâmetros



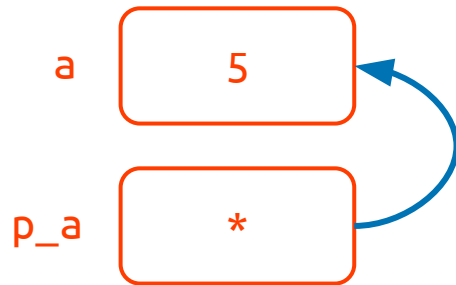
- Passagem por **valor**:
 - vale para valores dos tipos simples: `int`, `float` e `char`
 - vale também para *structs*
 - o parâmetro contém uma **cópia** do valor original
 - alterações no parâmetro não se refletem na variável original
- Passagem por **referência**:
 - vale para todos os tipos de vetores, inclusive *strings*
 - o parâmetro **aponta** para a variável original
 - alterações nos elementos do parâmetro têm efeito na variável original

Ponteiros

- Ponteiro é simplesmente uma referência para outra variável
 - Um ponteiro é **declarado** usando ***** e o tipo para que vai apontar
 - Ele deve ser **inicializado** com **&**, apontando para uma variável já existente

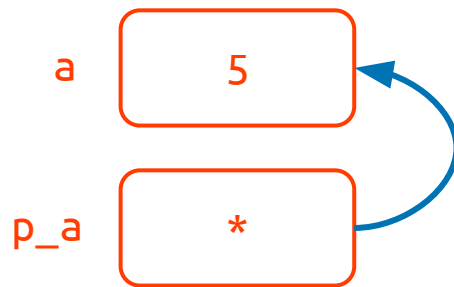
- Exemplo:

- `int a = 5;`
- `int *p_a = &a;`



- O valor apontado pelo ponteiro pode ser acessado usando *****
 - O ponteiro **tem** que apontar para uma variável
 - Recomenda-se o formato **(*p)** para evitar confusão com o operador de multiplicação
- Exemplo:

- `int a = 5;`
- `int *p_a = &a;`
- `int b = (*p_a);`
- `printf("%i\n", (*p_a));`



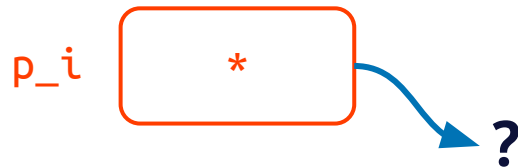
- Podemos usar ponteiros para passar variáveis de tipos simples por referência
 - Estas variáveis podem ser então modificadas pela função chamada
- Exemplo:
 - `void imprimir(int *p_i) {`
 - `printf("%i\n", (*p_i));`
 - `}`
 - `int main(void) {`
 - `int a = 2;`
 - `imprimir(&a);`

- Uma *string* pode ser representada como ponteiro para **char**
 - Funciona da mesma forma que um vetor de caracteres
- Exemplo:
 - `void func(char s1[], char *s2) {`
 - `printf("%s %s\n", s1, s2);`
 - `}`
 - `int main(void) {`
 - `char b[] = "bola";`
 - `char *c = "dado";`
 - `func(b, c);`

- Um vetor de strings precisa ser declarado combinando tanto * como []
- Exemplo:
 - `char *lista[] = {"Ana", "Beto", "Carla"};`
 - `char *nome = lista[0];`
 - `printf("%s\n", nome);`

- Podem ser omitidos, o que é o usual:
 - `int main()` {
- Se declarados, informam a quantidade total em `argc` e as *strings* de cada argumento no vetor `argv`:
 - `int main(int argc, char *argv[])` {
 - `for (int i = 0; i < argc; i++)` {
 - `printf("[%i] %s\n", i, argv[i]);`
 - `}`
 - `return 0;`
 - `}`

- Um ponteiro não inicializado:
 - Aponta para uma posição qualquer da memória
 - Certamente será **fonte de problemas!**
 - `int *p_i;`
- Devemos inicializar com **NULL**:
 - `int *p_i = NULL;`
- E assim testar depois se é válido:
 - `if (p_i != NULL) {`
 - `// ...`



Exercícios

- Crie uma função que recebe duas variáveis inteiras por referência (usando ponteiros) e que troca o valor das mesmas.
- Faça uma função que recebe um vetor de inteiros e seu tamanho como parâmetros. Esta função deve encontrar o menor e o maior valor presentes, e retorná-los através de dois parâmetros adicionais (ambos inteiros e passados por referência).

Perguntas?